MURL Chess Results: October 14, 2026

Minimax and MCTS on the 20 corrected positions

Akbar Aman
For Professors Evangelos Kobotis and Drew Shulman | Meeting: 8:30 PM, America/Chicago

What I tested and found

I tested my existing Python implementations of Minimax and Monte Carlo Tree Search (MCTS) on the 20 corrected positions, comparing their first moves with independently verified solutions. Minimax at depth 3 chose the correct first move in **6/20 positions**; MCTS did so in **9/60 runs** across three random seeds. My depth-5 Minimax attempts completed only **2/20 searches**; the other 18 reached the 20-second limit.

In everyday terms: I asked each program to choose the first move of a forced checkmate, then checked its answer. A correct first move does not by itself show that the program can finish the combination. A timeout means I stopped the search before it finished; I report its partial answer separately.

Why three moves can require five plies

A *ply* is one player's move. In my Minimax code, depth 3 means *my move, their reply, my next move*. A mate in three attacking moves can require *my move, their reply, my move, their reply, my mating move*: **five plies**. I therefore retained the requested three-ply runs and also attempted five-ply searches. These measure different amounts of look-ahead.

How I checked the positions and solutions

I used only the corrected `Positions2.pdf`, checking all 1,280 board squares, colors and side markers. Each board is stored as a *FEN*, a compact text description. All 20 passed `python-chess` validity checks; only case 3 is Black to move. Structural legality does not establish that a position occurred in the named historical game.

For the reference solutions, I used a separate exact AND/OR search to test every legal first move at 1, 3 and 5 plies. It found **one winning first move per position at five plies, and none at one or three**. The attacker needs *one* move that works, but that move must withstand *every* defensive reply. Formally, for legal moves $L(s)$ and successor s_m,

$$W(s,d) = \bigvee_{m \in L(s)} W(s_m, d-1) \quad \text{(attacker)}, \qquad W(s,d) = \bigwedge_{m \in L(s)} W(s_m, d-1) \quad \text{(defender)}.$$

Here W means a forced mate within the remaining d plies: defender checkmate is success; other endings or an exhausted horizon are failure. Saved mating strategies were also checked against every defensive reply. The verifier includes quiet moves and all promotions. Its search logic is separate, but it shares the programs' chess-rules library.

Search settings and measured results

Minimax looks ahead and assumes the opponent chooses the best reply. At its stopping point, my existing version mainly counts material (P=1, N=B=3, R=5, Q=9; mate=±10000). **MCTS** explores sampled continuations and uses their results to guide further exploration. I used 1,000 simulations per run, seeds 0/1/2, UCT exploration constant $\sqrt{2}$, and random rollouts capped at 40 plies. A seed makes a random run repeatable. MCTS uses a simulation budget, *not a fixed search depth*; its rollout cap is separate from the tree path already explored.

I kept the September implementations unchanged (commit `046a8d3`). Calls ran serially on Windows with Python 3.12.14 and chess 1.11.2. Each measured search had a 20-second limit; each verifier call had 120 seconds. Times include search overhead, excluding imports and output writing.

Method	Completed	Correct/completed	Timeouts	Median seconds†
Minimax, 3 plies	20/20	6/20	0	0.535
Minimax, 5 plies	2/20	2/2	18	8.195
MCTS, 1,000 simulations	60/60	9/60	0	2.724

†Completed searches only. The five-ply median represents just cases 16 and 18. These are not equal-time budgets or a general measure of playing strength.

Results for all 20 positions

Notation: K=king, Q=queen, R=rook, B=bishop, N=knight; x=capture, +=check, ==promotion. For example, Nxg7+ means a knight captures on g7 with check. **Bold** marks a correct selection; an asterisk marks a timeout's partial answer. MCTS columns identify the seed.

Case	Verified move	Minimax 3	Minimax 5	MCTS 0	MCTS 1	MCTS 2
1	Nxg7+	Nf4	Bf8*	Bb4	**Nxg7+**	g5
2	Bf5+	Be2+	Bxh7+*	fxe7	fxe7	Bf1+
3	Ne2+	Bc3	Kh7*	Rxg2	Nh1	Bc5
4	Qd8+	Qe3	**Qd8+***	Qxe4	Qxe4	a3
5	Rxg3	Nf3	Re7*	**Rxg3**	hxg3	**Rxg3**
6	Qc8+	**Qc8+**	**Qc8+***	Qxe6	Qxe6	Qxe6
7	Ng5	**Ng5**	**Ng5***	Kg2	Qe3	Qf3
8	Qxg8+	Qf8	Ne7*	Ne7	Qb7	Qf4
9	Rxg8+	Qd6	**Rxg8+***	Qf4	Qa6	Rb2
10	Rh8+	Qxh2+	**Rh8+***	Qc5	Qa6	Qf8
11	Rd8+	Qe8+	Ne7+*	Qxb8	Rxb4	Qe1
12	Rxg7+	Ne7	**Rxg7+***	Qb6	Qg1	Rd7
13	Nf6	Kxd1	**Nf6***	Kxd1	Kxd1	Kxd1
14	Nd6	hxg7+	Qxg7+*	Qe3	Kh2	Kf1
15	Rxa8	**Rxa8**	Be7*	**Rxa8**	**Rxa8**	**Rxa8**
16	Rf7	Kg4	**Rf7**	Rg1	Kf4	Ra1
17	d8=Q+	**d8=Q+**	Rh6*	Ra6	a8=Q+	Rf2
18	Rxg7	**Rxg7**	**Rxg7**	**Rxg7**	Kb1	**Rxg7**
19	Qg8+	**Qg8+**	**Qg8+***	d7	**Qg8+**	Qe2
20	c8=Q+	Rxa7	Re6*	Rxa7	Rxa7	Rxa7

What I conclude from these results

Limited look-ahead matters. Three-ply Minimax cannot see the full five-ply mating sequence. I interpret its six correct choices as first-move matches, not proof of the entire mate. MCTS's nine matches also do not prove a forced continuation. All 60 MCTS runs finished, but 89.6% of their 60,000 rollouts hit the cap and used a material estimate. Most simulated games therefore ended before a real outcome was available; their scores are not calibrated winning probabilities.

Deeper search was costly. Only cases 16 and 18 finished at five plies; both chose correctly. Eight timeout answers also matched, but I exclude them from completed-search success: 10 matching returned moves do not mean 10 completed solutions. I ran one timing trial per Minimax condition and three MCTS seeds. This small sample, desktop load and fixed run order limit broader comparisons.

My next step is to discuss the intended depth convention, then consider alpha-beta pruning and move ordering to reduce unnecessary Minimax search, and larger MCTS budgets. I have not run neural-network training; that remains the next stage.

Validation, assumptions, and reproducibility

Validation includes 16 passing tests, audits of all 160 raw records, and three exact fixed-seed MCTS replays. The reference search ruled out faster mates and other winning first moves; saved strategies check every defensive reply.

I assumed no castling or en-passant rights, clock zero and no earlier repetition history. All six relevant Black castling-right alternatives for cases 1 and 4 preserved the verified moves. Other diagrams permit neither castling nor immediate en-passant. Historical clocks and optional draw claims are outside this experiment.

I preserved the September baseline. The separate branch includes code, FENs, individual runtimes, search counts, raw logs, proof trees and reproduction commands.

Repository: ak23bar/murl-chess: october14-corrected-positions.

References: Corrected `Positions2.pdf`, four pages; September baseline and `references.bib` (AIMA Minimax, Kocsis–Szepesvári UCT, Browne et al. MCTS); python-chess core documentation, accessed October 14, 2026.